Semi-Autonomous PDF Accessibility Compliance Auditing Agent (AccessAudit) - Module Summary

Author: Aung Ze Ya | **Date:** November 5, 2026 | **Course:** MSc in Artificial Intelligence
Primary Frameworks: Python | OpenAI API (`gpt-4o-mini`) | Plan-and-Execute | Jupyter Notebook

Overview

The Problem. Public agencies, universities, and businesses frequently publish PDFs that are legally required to be accessible under the Americans with Disabilities Act (ADA) and related standards such as PDF/UA (International Organization for Standardization, 2019). However, manual accessibility auditing is slow, inconsistent, and often deferred. While automated checkers exist, they typically return raw, technical output that a non-specialist cannot interpret, prioritize, or act upon.

The Solution. This project implements **AccessAudit**, a semi-autonomous agentic AI system designed to bridge the gap between raw accessibility checkers and human decision-making. AccessAudit ingests a user-supplied PDF, runs a conditionally ordered sequence of accessibility checks, interprets the results, and produces a structured, advisory remediation report.

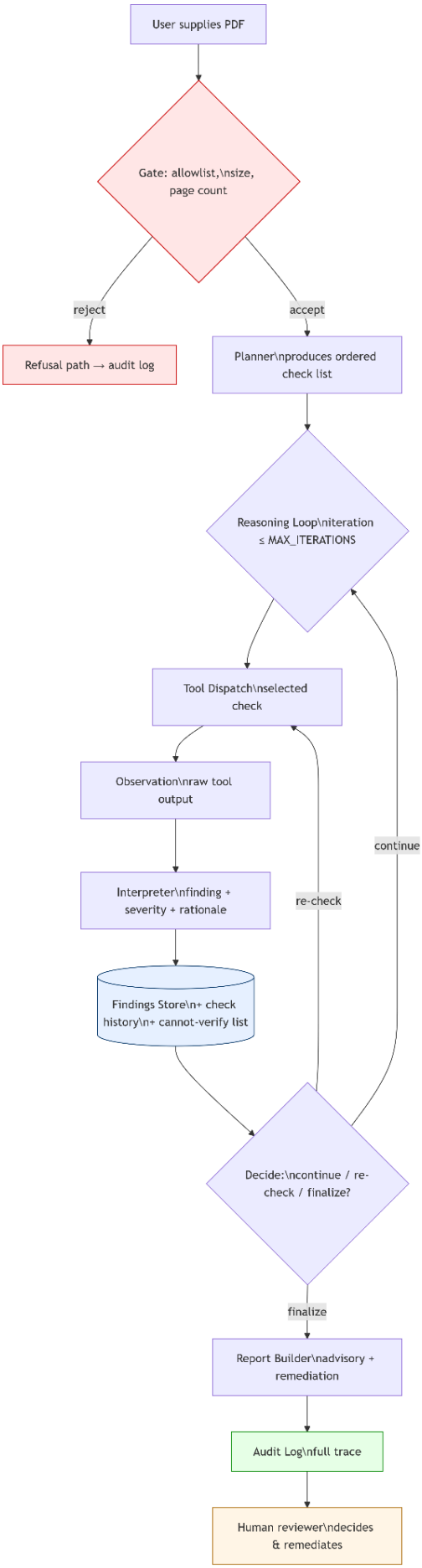
Agentic Approach. The system is built as a single-agent **Plan-and-Execute** workflow. It uses a bounded reasoning loop to decide which checks to run and in what order, maintaining a memory of its findings across steps. It utilizes six deterministic tools (backed by **`pypdf`**, **`pymupdf`**, and **`reportlab`**) to inspect the document's internal structure, metadata, text extraction, alt-text presence, heading hierarchy, and link annotations. An LLM (**`gpt-4o-mini` via the OpenAI API**) is integrated as a bounded interpreter, used only to reword technical findings into plain language without altering the underlying verdicts.

Scope and Boundaries. AccessAudit is intentionally designed as a **read-only, advisory system**. It does not modify the input PDF, nor does it certify legal compliance. It operates within strict safety boundaries: a file-type allowlist (PDF only), size and page-count caps, a maximum iteration limit, and an explicit "cannot verify" mechanism for checks beyond its capabilities.

Core Design Principle. The agent embodies a strict autonomy boundary: **AccessAudit autonomously inspects and advises, but a human reviewer retains the authority to decide what to fix and how.** By surfacing uncertainty explicitly rather than manufacturing false confidence, the system provides actionable intelligence while preventing the dangerous "compliance theater" of treating an automated scan as a legal certificate.

System Architecture and Design

AccessAudit is built as a **single-agent Plan-and-Execute workflow**. This architecture was chosen over a multi-agent system because the accessibility checks are sequential and depend on shared state (e.g., checking for alt-text is meaningless if the document is untagged). Multi-agent coordination would introduce unnecessary handoff overhead and failure modes for a task where a single agent can maintain a cohesive findings store (Wooldridge, 2009).


User supplies PDF
Gate: allowlist,\nsize, page count
reject
accept
Refusal path → audit log
Planner\nproduces ordered check list
Reasoning Loop\niteration ≤ MAX_ITERATIONS
Tool Dispatch\nselected check
Observation\nraw tool output
Interpreter\nfinding + severity + rationale
Findings Store\n+ check history\n+ cannot-verify list
re-check
continue
Decide:\ncontinue / re-check / finalize?
finalize
Report Builder\nadvisory + remediation
Audit Log\nfull trace
Human reviewer\ndecides & remediates

Core Components

The system consists of five interacting components designed to ensure transparency, reliability, and safety:

1. **The Agent Persona & System Prompt:** The agent is instructed to act as a "cautious, evidence-based PDF accessibility auditor." The system prompt enforces hard rules: it must never claim legal compliance, must use a fixed severity vocabulary (Critical, Major, Minor, Info), and must record "cannot verify" if a tool cannot confirm a check.
2. **Memory & State Handling:** The agent maintains a `SessionState` object that acts as its working memory. This state tracks the document hash (for traceability), the iteration count, an ordered history of checks run, a structured findings store (severity, rationale, remediation), a list of "cannot verify" items, and a full audit log of every decision made.
3. **Tool Registry:** The agent has access to exactly six read-only tools, which it dispatches based on its plan. These are: `check_tagged_pdf`, `get_metadata`, `check_text_extractability`, `inventory_alt_text`, `check_heading_order`, and `inspect_links`.
4. **The Reasoning Loop:** At the heart of the system is a bounded loop (max 12 iterations). The Planner selects a tool based on prior observations, the tool executes, the Interpreter converts the raw output into a structured Finding, and the agent decides whether to continue, re-check a failure, or finalize.
5. **Report Builder:** Once the loop finalizes, the agent generates a report (in Markdown, DataFrame, and terminal formats) that includes an **advisory-only verdict**, a severity summary, detailed findings, and prioritized remediation steps.

Design Tradeoffs

- **Plan-and-Execute vs. Free-form ReAct:** A free-form ReAct (Reasoning and Acting) loop (Yao et al., 2023) allows an LLM to dynamically choose any tool at any time. However, accessibility auditing is a known, finite checklist with strict conditional ordering. Plan-and-Execute was chosen because it makes the workflow deterministic and reproducible; the same PDF will yield the exact same audit path, which is essential for compliance domains.
- **Bounded LLM Authority:** The LLM (`gpt-4o-mini`) is not allowed to make decisions or assign severities. It is strictly used as an interpreter to reword the raw tool output into plain English. If the LLM fails or is unavailable, the system falls back to a deterministic, rule-based interpreter. This prevents the LLM from hallucinating a "pass" on a failed check.

Decision Logic and Behavior

AccessAudit does not use a free-form conversational loop; instead, it relies on a deterministic planner (`plan_next_action`) combined with rule-based interpretation and bounded LLM summarization. This ensures that the agent's behavior is highly predictable, auditable, and reproducible across different runs.

1. How the Agent Makes Decisions

The agent’s decision-making process operates at three levels:

- **Plan Level (What to do next):** The agent always runs `check_tagged_pdf` and `get_metadata` first. If the PDF is **untagged**, the planner immediately halts the sequence and finalizes. It skips the alt-text, heading, and link checks entirely, because those checks rely on the structural tag tree and would yield meaningless results on an untagged document. If the document is tagged, the planner proceeds sequentially through the remaining registered tools.
- **Execution Level (Re-check Gate):** When a check fails, the agent does not immediately finalize the failure. The `decide_recheck` logic allows exactly one re-execution of that specific tool to confirm stability. If the tool fails twice, it is recorded as a definitive fail. If the re-check disagrees with the first result, the finding is downgraded to "cannot verify" and flagged for human review.
- **Interpretation Level (Severity and Rationale):** Severity is hardcoded in the tool registry (e.g., untagged PDFs

are always "Critical," missing alt-text is "Major"). The LLM is invoked *only* to reword the rationale into plain language; it cannot alter the severity, the pass/fail state, or the remediation instructions.

2. How the Agent Responds to Different Inputs

The system was executed on five distinct PDF fixtures (§7 of the notebook), demonstrating clearly differentiated behavior:

- **On a Compliant Document (F1):** The agent ran all six checks, passed five, and returned "cannot verify" on the link inspection. It finalized with zero Major or Critical findings, demonstrating a clean happy path.
- **On an Untagged/Image-Only Document (F2):** The agent failed the mandatory `check_tagged_pdf` and `get_metadata` checks. It re-checked both (taking 4 iterations total). Once both failures were confirmed, it halted the plan, skipping the remaining dependent tools. It correctly assigned a "Critical" severity to the untagged structure and a "Major" severity to the missing language metadata.
- **On a Missing Alt-Text Document (F3):** The agent passed the structural checks but failed the `inventory_alt_text` check. It triggered a re-check, which agreed with the first failure, and finalized with a "Major" finding.
- **On a Wrong Language Document (F4):** The agent detected the tag structure successfully but failed the metadata check because the document declared `/Lang=fr-FR` instead of English. It correctly flagged this as a "Major" metadata failure.
- **On Invalid Input (F5):** The agent's input gate intercepted the file (a text file renamed to `.pdf`) before any tool ran. It refused the audit, recorded the refusal reason in the audit log, and finalized immediately with zero iterations.

3. Defensive Decision-Making

The agent's logic is heavily biased toward safety and transparency:

- **Fail-Soft Execution:** If a tool raises an unexpected exception (e.g., a corrupted PDF object), the wrapper catches it and converts it into a "cannot verify" result rather than crashing the loop.
- **Explicit Uncertainty:** If a tool lacks the capability to confirm a result (such as `inspect_links` being unable to extract the visible text of a link annotation), it explicitly returns "cannot verify" rather than passing it by default.
- **Hard Caps:** The loop is strictly bounded by a maximum of 6 distinct checks and 12 iterations. This prevents runaway loops and ensures the agent cannot invent new tools or get stuck in a re-check spiral.

Safety, Reliability, and Transparency

Because AccessAudit operates in a legal compliance domain (ADA and PDF/UA), the system is designed with strict safeguards to prevent harm, ensure reliability, and maintain transparency. These measures are enforced structurally in the code, not merely stated as instructions to the LLM.

1. Safety and Reliability Safeguards

- **Input Gate (Allowlist & Caps):** Before any parsing tool touches the file, the agent runs a validation gate. It enforces a file-type allowlist (`.pdf` only), a size cap (25 MB), a page-count cap (200 pages), and a magic-byte check (ensuring the file header starts with `%PDF-`). This prevents the agent from attempting to process malicious or out-of-scope files.
- **Read-Only Operations:** Every tool is strictly read-only. The agent never modifies the input PDF, nor does it write to the user's filesystem during the audit.
- **Hard-Coded Loop Caps:** The reasoning loop is bounded by `MAX_CHECKS` (6) and `MAX_ITERATIONS` (12). This prevents runaway execution or infinite loops if the agent gets confused.
- **Fail-Soft Tool Wrappers:** If a tool encounters an unexpected exception (e.g., a malformed PDF object), the system catches the error and records it as a "cannot verify" finding rather than crashing the entire agent.
- **Re-Check Gate:** A single failure is not immediately accepted as definitive. The agent schedules exactly one re-check to confirm stability. If the tool disagrees with its first result, the agent downgrades the finding to

"cannot verify" rather than emitting a false positive or false negative.

2. Transparency Measures

- **The Audit Log:** Every single iteration of the reasoning loop is recorded in a structured audit log. This log captures the timestamp, the iteration number, the tool used, the raw observation, the decision made (continue, re-check, or finalize), and the rationale. A human reviewer can reconstruct exactly how the agent arrived at its conclusion.
- **The "Cannot Verify" Fallback:** The agent is explicitly designed to admit its limitations. If the tooling cannot confirm a check (e.g., `inspect_links` cannot extract the visible text of a link annotation), the agent records "cannot verify" rather than a "pass." The report builder displays these explicitly in a dedicated section so they are not silently dropped.
- **The Advisory-Only Verdict:** Every generated report (whether in JSON, Markdown, or terminal display) contains a mandatory verdict block stating: *"Advisory findings only. This is not a legal ADA or PDF/UA compliance certification. A human reviewer must decide what to remediate."* This structurally prevents the agent's output from being mistaken for a legal certificate.
- **Bounded LLM Authority:** The LLM (`gpt-4o-mini`) is restricted from making decisions. It cannot change a "fail" to a "pass," assign a severity, or perform remediation. It is used strictly as a summarizer to reword raw tool output into plain language. If the LLM fails or times out, the system falls back to a deterministic rule-based rationale.

Observed Behavior and Limitations

The agent's behavior was evaluated across five distinct PDF fixtures, with the results and execution traces detailed in §7 and §8 of the accompanying Jupyter Notebook. The system successfully matched the expected outcomes across all five scenarios, but the runs also surfaced concrete limitations that are critical to the system's honest evaluation.

1. Observed Behavior (What Worked)

The execution traces demonstrate that the agent behaves as a dynamic planner rather than a static script.

- **Dependency-Driven Planning:** On the untagged fixture (`F2`), the agent failed the initial structural check and stopped after four iterations (two mandatory checks plus their re-checks), rather than attempting to run all six checks. This proves the agent actively reasons about its own plan and skips dependent tools when the foundational prerequisite (a tag tree) is missing.
- **The Re-check Gate:** On fixtures `F2`, `F3`, and `F4`, the agent experienced a hard failure, triggered a re-check, and only finalized the finding once the re-check agreed. This mechanism successfully prevents transient parsing errors from being reported as definitive compliance failures.
- **Safety Boundaries:** On the malformed fixture (`F5`), the agent's input gate successfully refused the file before any parsing occurred (`iterations=0`), recorded the precise refusal reason in the audit log, and surfaced the refusal in the final report.

2. Genuine Limitations and Failure Cases

Despite the expected behaviors, the runs exposed five specific limitations (labeled `F-1` through `F-5` in the notebook):

- **Tool Limitation (F-3): `inspect_links` cannot verify text.** On all tagged, text-bearing fixtures (`F1`, `F3`, `F4`), the link-inspection tool consistently returned *"cannot verify — annotations carry no /Contents; visible text not inspectable."* Because `pypdf` extracts the link annotation's position but not the visible text in the page content stream, the agent could not definitively confirm if the link text was descriptive. Rather than manufacturing a false pass, the agent honestly reported uncertainty.
- **LLM Boundary (F-4): Rewording adds unverified interpretation.** On `F3`, the raw tool output was a simple factual statement: *"1 of 1 images lack /Alt."* The LLM reworded this into: *"The PDF contains one image that lacks alternative text, which is essential for accessibility."* While the severity (Major) and verdict (Fail) remained correct, the clause *"which is essential for accessibility"* is an evaluative claim not present in the raw data.

Bounding what the LLM may *decide* does not automatically bound what it may *say*.

- **Design Divergence (F-2): Loop iterations exceeded expectation.** On F2, the agent ran four iterations instead of the expected two. The re-check gate fired on *both* mandatory failures (structure and metadata) before the early-exit rule could halt the plan. The agent's behavior was correct and more rigorous, but it deviated from the initial design intent.
- **Heuristic Weakness: `check_heading_order` cannot verify tag trees.** The heading tool uses a font-size heuristic (checking if sizes decrease monotonically) rather than traversing the actual tag tree. On F2, it returned *"cannot verify — no heading-sized text detected."* A font-size heuristic is insufficient for verifying the logical reading order required by PDF/UA standards (International Organization for Standardization, 2019).
- **Scope Limitation (F-5): Active content not scanned.** The input gate checks extension, size, header, and page count, but it does not inspect the PDF for `/JavaScript` or `/EmbeddedFiles`. This is a scope limitation, mitigated by the fact that all tools open the document read-only and do not execute embedded scripts.

Ethical and Responsible Use Considerations

The deployment of AI in legal compliance domains carries significant ethical risks, particularly regarding autonomy, accountability, and the potential for misuse. Because AccessAudit interacts with ADA and PDF/UA accessibility standards, its ethical framework is grounded in the specific behaviors observed during evaluation.

1. The Difference Between "Cannot Verify" and "Pass" (Case F-3)

The most consequential ethical decision in this system's design is how it handles uncertainty. During the audits of F1, F3, and F4, the `inspect_links` tool returned *"cannot verify"* because it could not extract the visible text of the link annotations. A weaker design might have treated "no obvious problem detected" as a "pass." However, doing so would produce a false sense of compliance for screen-reader users. The agent's ethical commitment is to surface uncertainty explicitly: it never conflates "cannot verify" with "pass," ensuring that human reviewers know exactly what was and was not checked.

2. The Limits of Bounded LLMs (Case F-4)

The system uses a bounded LLM (`gpt-4o-mini`) to reword technical findings into plain language, but it is structurally prohibited from changing the underlying severity or verdict. However, the F3 run revealed a subtle ethical boundary: the raw tool output was *"1 of 1 images lack /Alt,"* but the LLM reworded it to include the phrase *"which is essential for accessibility."* This evaluative claim was not present in the raw data. The lesson is that bounding what an LLM may *decide* does not fully bound what it may *say* (Yao et al., 2023). Any production deployment must account for lexical drift and consider dropping the LLM path entirely when the deterministic rule-based rationale is already sufficient.

3. Compliance Theater and the Advisory Verdict (Cases F-3, F-5)

The system could be misused as a "compliance certificate." An organization might run a batch of PDFs, see zero Critical or Major findings (because the automated tools missed issues or returned "cannot verify"), and falsely claim ADA compliance—a phenomenon known as compliance theater (Liu & Wang, 2023). To mitigate this, every output format generated by the Report Builder (§5) includes a mandatory `verdict` block explicitly stating: *"Advisory findings only. This is not a legal ADA or PDF/UA compliance certification."* This structural intervention makes misuse difficult, though not impossible.

4. Accountability in the Re-check Gate (Case F-2)

On fixture F2, the agent ran four iterations instead of two because two mandatory checks failed and each was re-verified. This is not a fault; it is the accountability mechanism working as intended. The agent did not accept a single observed failure as definitive and instead re-verified the result to ensure stability. A design that trusted a single tool result would run faster but would be less rigorous and more prone to false positives. The extra iterations are the ethical cost of verifiability.

5. The Human-in-the-Loop Boundary

Ultimately, the system is designed as *semi-autonomous*, not fully autonomous. It inspects and advises, but it never certifies compliance and never modifies the document. The ethical responsibility for what happens after the report is generated—the actual remediation of accessibility barriers—rests entirely with the human reviewer (World Wide Web Consortium, 2023). The agent's role is strictly to provide the evidence required for a human to make an informed decision.

Future Improvements

While AccessAudit successfully demonstrates a bounded, semi-autonomous auditing workflow, the failure cases and limitations identified in this report provide a clear roadmap for future enhancements. The following improvements would address the system's current technical and scope limitations.

1. Upgrading Link Inspection (Resolving Case F-3)

The most significant technical limitation observed was that `inspect_links` consistently returned "cannot verify" because `pypdf` cannot correlate a link annotation's position to the visible text in the page content stream. A future iteration should integrate `pymupdf`'s layout analysis capabilities to clip the page to the annotation's rectangle and extract the text. This would allow the agent to compare the extracted text against a non-descriptive phrase list (e.g., "click here," "read more"), upgrading the check from a "cannot verify" to a definitive pass or fail.

2. Mitigating LLM Lexical Drift (Resolving Case F-4)

During the `F3` run, the LLM reworded a raw finding to include an unverified evaluative claim ("which is essential for accessibility"). To prevent this, future work should implement a lexical-drift check. This could involve a token-overlap threshold that compares the LLM's reworded rationale against the original tool message, flagging any additions. Alternatively, the simplest and most robust fix is to disable the LLM path entirely for the six deterministic tools, as their raw rule-based outputs are already accurate enough for a technical report.

3. Expanding the Input Gate (Resolving Case F-5)

Currently, the input gate checks file extension, size, PDF header, and page count. It does not scan for embedded active content such as `/JavaScript`, `/EmbeddedFiles`, or `/OpenAction` keys. Future improvements should include a `pikepdf`-based pre-scan that traverses the object tree. If active content is detected, the agent should classify the file as "needs human review before audit" and refuse to process it by default, further tightening the safety boundary.

4. True Tag-Tree Traversal for Logical Reading Order

The `check_heading_order` tool currently relies on a font-size heuristic (checking if headings decrease monotonically) rather than traversing the actual PDF tag tree. While this catches obvious visual anomalies, it cannot verify the true logical reading order required by PDF/UA standards. Future iterations should use `pikepdf` to walk the `/StructTreeRoot` and validate the exact hierarchy of heading tags (H1, H2, H3), upgrading the heuristic to a deterministic compliance check.

5. Knowledge Base and Remediation Generation

Currently, the agent provides a static remediation hint for each failed check. A more advanced version could integrate a retrieval-augmented generation (RAG) system linked to a database of accessibility guidelines (such as WCAG 2.2 or ISO 14289). When a failure is detected (e.g., missing alt-text), the agent could query the knowledge base to generate context-specific remediation instructions or even draft the missing alt-text for a human to review.

References

American with Disabilities Act of 1990, 42 U.S.C. § 12101 *et seq.* (1990). https://www.ada.gov/law-and-regs/ada/

Adobe Systems Incorporated. (2023). *PDF accessibility overview*. Adobe. https://www.adobe.com/accessibility/pdf/pdf-accessibility-overview.html

International Organization for Standardization. (2019). *Document management applications — Electronic document file format enhancement for accessibility — Part 1: Use of ISO 32000-1 (PDF/UA-1)* (ISO Standard No. 14289-1:2014). https://www.iso.org/standard/64599.html

Liu, X., & Wang, Y. (2023). Accessibility barriers in digital documents: A systematic review of automated PDF auditing. *Universal Access in the Information Society*, 22(4), 1013–1032. https://doi.org/10.1007/s10209-022-00918-8

World Wide Web Consortium. (2023). *Web Content Accessibility Guidelines (WCAG) 2.2*. W3C Recommendation. https://www.w3.org/TR/WCAG22/

Wooldridge, M. (2009). *An introduction to multiagent systems* (2nd ed.). Wiley.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. In *Proceedings of the 11th International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/2210.03629